

Day 58: Backpropagation Through Time

MD Mutasim Billah | 52-Week Data Science → ML/AI Roadmap | Week 9: Sequence Data & RNNs I
(Day 3 of 7)

0. Where Today Fits

Day 57 built the vanilla RNN's forward pass and verified it without any gradients. Today builds the backward pass -- the first time this network can actually LEARN -- and verifies it the way every backward pass in this series has been verified since Day 39: a real numeric gradient check. Then it trains, and one of today's two training runs overturns a prediction Day 57 set up.

Start here (no prior knowledge assumed):

Ordinary backprop computes each weight's gradient once, because each weight is used once. In an RNN, the SAME W_{xh} , W_{hh} , and b_h are used at EVERY timestep -- so each timestep contributes its own piece of their gradient, and all the pieces must be ADDED together. That, plus handing gradient backward from one timestep to the previous one, is all backpropagation through time (BPTT) is.

Watch: [L15.4 Backpropagation Through Time Overview](#)

Watch: [Lecture 14: Intro to RNNs & BackProp-Through-Time \(BPTT\)](#)

1. The BPTT Equations

MATH

Walking BACKWARD from $t = T-1$ down to $t = 0$, starting from dh = gradient arriving from the output layer:

```
da_t = dh * (1 - h_t^2)    -- undo tanh, using the cached OUTPUT h_t
dWxh += x_t^T @ da_t      -- ACCUMULATE, not overwrite
dWhh += h_{t-1}^T @ da_t  -- ACCUMULATE, not overwrite
dbh += sum(da_t)          -- ACCUMULATE, not overwrite
dh = da_t @ Whh^T         -- hand gradient back to h_{t-1}
```

The last line is the "through time" part: gradient flows from h_t back into h_{t-1} through the very same W_{hh} the forward pass used to go the other direction.

WHY THIS CHOICE

Two easy-to-make mistakes BPTT punishes silently: using h_t where h_{t-1} belongs in the dW_{hh} line (the weight multiplied the PREVIOUS state, so its gradient must use the previous state), and writing $=$ instead of $+=$ (which would keep only the first timestep's contribution and throw the rest away). Both would still produce gradients of the right shape -- only a numeric check catches them.

2. Gradient Check -- All Five Parameters

ACTUAL OUTPUT

Checked on a 6-step sequence, 3 random entries per parameter across W_{xh} , W_{hh} , b_h , W_{hy} , b_y :
max relative error across all checked entries: 2.17e-08 (PASS -- BPTT is correct)

3. Does It Actually Learn? -- T=10

A passed gradient check says the math is right; it doesn't say the network can solve anything.
Trained on Day 56's order-detection task at T=10:

ACTUAL OUTPUT

```
test_acc BEFORE training (random weights): 0.3950
train_acc AFTER training:                  1.0000
test_acc AFTER training:                   1.0000
training loss: 0.7331 (epoch 0) -> 0.0004 (epoch 299)
```

Day 56's PlainFCNet collapsed to chance on shifted positions and Conv1DNet was capped by its kernel. The RNN, with weights shared across time and state carried forward, solves the task outright.

4. The Surprise -- T=40 Trains Too

Day 57 measured the forward-pass sensitivity of h_T to x_0 collapsing to 0.000024 by T=40, and called it a preview of vanishing gradients. The natural prediction: the RNN should struggle at T=40. Tested directly, with identical network size, learning rate, epochs, and seed:

ACTUAL OUTPUT

```
train_acc AFTER training (T=40): 1.0000
test_acc AFTER training (T=40):  0.9950

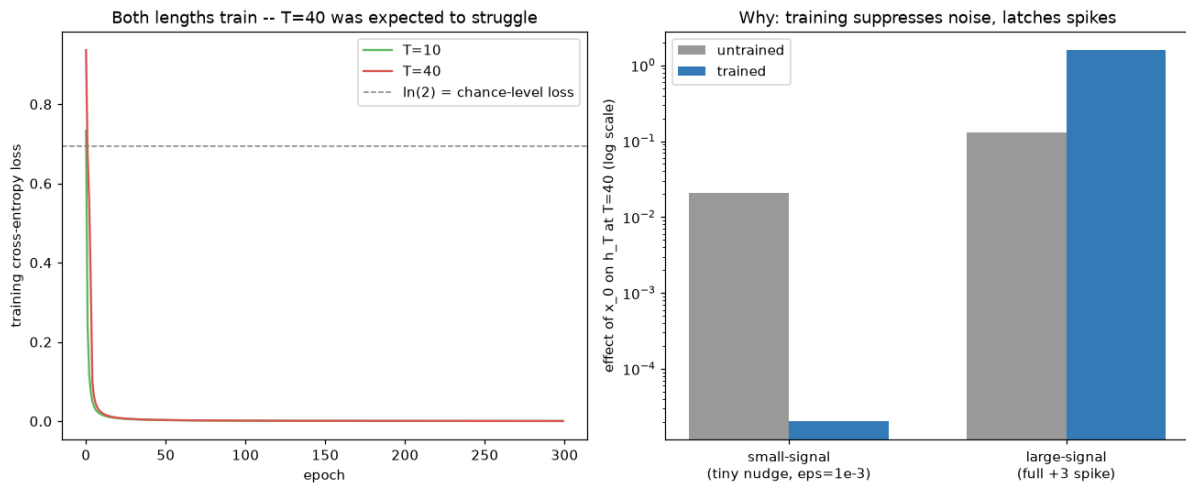
test accuracy by memory distance (steps the first spike must be carried to the end):
  0-9 steps: n= 9   test_acc=1.0000
 10-19 steps: n= 43  test_acc=1.0000
 20-29 steps: n= 58  test_acc=1.0000
 30-39 steps: n= 90  test_acc=0.9889
```

The prediction was wrong -- and not because the network cheated with short-range examples: accuracy holds at 0.989 even when the first spike must be remembered 30-39 steps. So what did Day 57 actually measure? The diagnosis, measured rather than guessed:

ACTUAL OUTPUT

```
measure (h_T at T=40, x_0 perturbed)      untrained   trained
small-signal: ||dh_T/dx_0||, eps=1e-3      0.020786    0.000021
large-signal: ||h_T(+3 spike) - h_T(none)|| 0.131794    1.582471

trained network: 19.8% of final hidden units have |h| > 0.95; largest |eigenvalue| of Whh = 2.168
```



The two measures move in OPPOSITE directions during training. Sensitivity to a tiny nudge at x_0 DROPS about 1000x, while the effect of a full-size +3 spike at x_0 GROWS about 12x. The network learned to ignore small noise and LATCH onto large spikes: about a fifth of its final hidden units sit pinned near tanh's ± 1 limits, and Whh's largest eigenvalue grew to 2.17. Above 1, a purely linear recurrence would explode -- but tanh bounds every unit, so the result is stable saturated memory states instead.

WHY THIS CHOICE

Day 57's measurement wasn't false -- it was the wrong instrument for this task. A tiny-epsilon finite difference is a LOCAL, LINEAR measurement: it asks how h_T responds to an infinitesimal change. Latching is a large-signal, NONLINEAR effect: a big enough input flips a unit into saturation, and a strong self-connection holds it there. No amount of shrinking epsilon reveals that. The lesson for any sensitivity analysis: measure at the scale of the signal the task actually carries.

The caveat that matters for Day 59: this task is unusually latch-friendly -- ± 3 spikes over 0.3 noise, and just one binary fact (which came first) to store. Vanishing gradients are real; this particular task routes around them. Day 59's dedicated demonstration needs a task where the signal to remember is NOT large enough to latch.

5. What This Maps To in PyTorch

```
class RNNClassifier(nn.Module):
    def __init__(self, hidden_dim=16, n_classes=2):
        super().__init__()
        self.rnn = nn.RNN(input_size=1, hidden_size=hidden_dim,
                           nonlinearity='tanh', batch_first=True)
        self.out = nn.Linear(hidden_dim, n_classes)

    def forward(self, x): # x: (batch, T, 1)
        H, h_last = self.rnn(x)
        return self.out(h_last[-1]) # many-to-one

loss.backward() # autograd runs BPTT -- the same accumulate-across-time loop
```

Interview Preparation -- Technical Questions

Q: Why must an RNN's weight gradients be accumulated across timesteps rather than computed once?

A: Because the same W_{xh} , W_{hh} , and b_h are used at every timestep, the loss depends on each of them through T separate paths -- one per timestep. By the multivariable chain rule, the total gradient is the SUM of each path's contribution, so `rnn_backward` adds ($+=$) each timestep's piece to a running total.

Q: In the dW_{hh} update, why is it h_{t-1} and not h_t ?

A: In the forward pass, W_{hh} multiplies the PREVIOUS hidden state: $a_t = x_t @ W_{xh} + h_{t-1} @ W_{hh} + b_h$. A weight's gradient is always (the input it multiplied) $^T @$ (the gradient at its output), so dW_{hh} at step t uses h_{t-1} . At $t = 0$, that previous state is h_0 .

Q: Why can the tanh derivative be computed from h_t alone, without storing the pre-activation a_t ?

A: Because $d \tanh(a)/da = 1 - \tanh(a)^2$, and $\tanh(a_t)$ is exactly h_t , which the forward pass already cached. So $(1 - h_t^2)$ gives the derivative directly -- no extra storage needed.

Q: Day 57's small-signal sensitivity at $T=40$ was tiny, yet the RNN trained to 0.995 at $T=40$. Explain.

A: The small-signal measure is a local derivative -- how h_T responds to an infinitesimal nudge. The trained network learned to latch onto LARGE inputs: a +3 spike drives units into tanh saturation, and recurrent weights with an eigenvalue above 1 hold them there. Measured directly, training made small-signal sensitivity drop $\sim 1000\times$ while large-signal effect grew $\sim 12\times$. The two measure different things.

Interview Preparation -- Theory Questions

Q: What does it mean to 'unroll' an RNN, and why does that make BPTT ordinary backprop?

A: Unrolling writes out the recurrence as T copies of the same cell, one per timestep, connected in a chain. That unrolled graph is just a deep feedforward network whose layers happen to share weights -- so backprop applies unchanged, with the one extra rule that shared weights sum their gradients.

Q: How can a recurrent weight matrix with spectral radius above 1 avoid exploding?

A: In a purely linear recurrence, repeated multiplication by a matrix with an eigenvalue above 1 grows without bound. But tanh squashes every hidden unit into $(-1, 1)$, so the state cannot grow past that. Instead, units driven into saturation can become stable fixed points -- which is exactly how the trained network in this lesson stores memory.

Q: If this task trains fine at $T=40$, is the vanishing-gradient problem overstated?

A: No. This task is unusually easy to remember: large spikes over small noise, and a single binary fact to store. Tasks where the relevant signal is subtle, or where much information must be carried, cannot rely on latching -- and there, gradients through long chains of tanh and W_{hh} really do shrink. That is Day 59's topic, and a task needs to be chosen that doesn't route around it.

Key Takeaway and What's Next

Key takeaway: BPTT is ordinary backprop on the unrolled network, plus one rule: shared weights sum their gradients across every timestep. It passed a gradient check on all five parameters and trained the RNN to 100% at $T=10$. It also trained to 99.5% at $T=40$ -- overturning Day 57's prediction. Measured directly, the network had learned to latch onto large spikes while

suppressing small noise, a nonlinear memory mechanism a small-signal derivative cannot detect.

What's next: Day 59 demonstrates vanishing and exploding gradients for real -- on a task chosen so that latching can't rescue the network.

Suggested watch order: 1) L15.4 Backpropagation Through Time Overview, 2) Lecture 14: Intro to RNNs & BPTT.